START HERE

Understand your Swara Player

Python syntax, MediaPipe commands, and every line of your working program.



You already understand the goal: show fingers and hear a note. This guide teaches how the actual statements make that happen. It starts with the punctuation and vocabulary, then follows the saved program from top to bottom.

No previous API knowledge assumed
A function name such as `HandLandmarkerOptions` is a name supplied by a library. You learn what input it accepts and what it returns; you do not have to invent or guess the name.

What is inside

You will learn	How it is explained
Read Python statements	Small examples of dots, brackets, calls, names, and indentation.
Understand MediaPipe	A labelled hand, coordinates, settings, images, and detector results.
Follow the complete program	Every executable source line is covered, with examples of its values.
Build and check it	Milestones, troubleshooting, practice answers, and the complete numbered source.

Based on the simplified 119-line `main.py` in your local project. Prepared 29 October 2026. The program itself has not been changed while making this guide.

READING ROUTE

How to use this guide

Keep your code beside the PDF. Read a small block, trace its values, then run it.

Pages	Read these when you want to...
3-7	Understand Python words and symbols before reading library calls.
8-10	See the program flow, numbered hand points, and image coordinates.
11-21	Understand the saved program, statement by statement.
22-23	Build the webcam and hand-detection milestones in order.
24-25	Follow a complete example and test your understanding.
26-27	Find an error or look up an unfamiliar word.
28-30	Read the complete source with the original line numbers.
31	Find the supplied material and official references.

For every statement, ask three questions

1. What value goes in?
2. What action or calculation happens?
3. What value comes back, and where is it saved?

```
success, frame = cap.read()
# Input: an already-open camera connection named cap.
# Action: read one image.
# Output: a success flag and an image, saved separately.
```

The two kinds of code in this PDF
Boxes labelled with source line numbers match the saved program. Other small examples explain syntax or show an equivalent longer form; they are not extra code to paste into the program.

The guide uses the current saved code as the reference. Your student notes sometimes use `time.time()` or a handedness label. The small differences in this version are explained on pages 15 and 18.

FOUNDATION 1

Values, variables, and comparisons

A variable is a name that lets you use a value again.

```
count = 3
name = "Ga"
success = True
cap = None
```

Example	Read it as
`count = 3`	Save the whole number 3 under the name count.
`name = "Ga"`	Save the text Ga. Quotes identify text.
`success = True`	Save a yes/no value. True means yes; False means no.
`cap = None`	Save a special value meaning that no camera object is stored yet.

Python evaluates the right side of an assignment first, then saves the result using the name on the left. `count = count + 1` reads the current count, adds one, and replaces the stored value.

```
count = 3
count += 1       # count is now 4
count == 4       # True: are the values equal?
count != 3       # True: are the values different?
count < 5        # True: is count smaller than 5?
```

Do not mix up = and ==
= stores a value. == asks whether two values are equal. != asks whether they are different. The result of a comparison is True or False.

Names are case-sensitive: `MODEL_PATH` and `model_path` are different names. Uppercase in this project is a convention for settings that we intend to keep fixed; Python does not enforce that convention.

A line beginning with # is a comment. It explains the code to a reader. Blank lines separate ideas. Neither comments nor blank lines perform an action.

FOUNDATION 2

Dots, parentheses, and returned values

Read a command from its owner to its action, then identify the inputs.

```
frame = cv2.flip(frame, 1)
```

Piece	Meaning
`cv2`	The imported OpenCV library.
`.`	Access a name belonging to that library or object.
`flip`	The function we want to use.
`(frame, 1)`	Call it with two inputs: an image and the flip direction.
`frame =`	Save the returned image under the name frame.

Parentheses after a callable name make the call happen. A comma separates inputs. A function can return a value that another statement can use.

```
result.hand_landmarks    # Read stored information.
landmarker.close()      # Run an action with no extra inputs.
```

An object can hold information and offer actions. A named piece of information is an attribute. A function attached to an object is a method. Here, `hand_landmarks` is information in result; `close()` is an action offered by landmarker.

What does creating an object mean?

```
base_options = mp_python.BaseOptions(
    model_asset_path=MODEL_PATH
)
```

`BaseOptions` is a class supplied by MediaPipe. Calling it creates one settings object. The variable `base_options` then refers to that object. The later `HandLandmarker` creation produces a detector object with a different job.

> **Names you choose and names the library chooses**
> You may rename your variable `landmarker` to `detector` if you update every use. You must keep the library method name `detect_for_video` as MediaPipe defines it.

FOUNDATION 3

Why does base_options appear twice?

A named input and a variable can have the same spelling but different jobs.

```
base_options = mp_python.BaseOptions(
    model_asset_path=MODEL_PATH
)

options = mp_vision.HandLandmarkerOptions(
    base_options=base_options,
    running_mode=mp_vision.RunningMode.VIDEO,
    num_hands=2,
)
```

Inside the call	Left of =	Right of =
`base_options=base_options`	Setting name expected by MediaPipe.	Your variable containing model settings.
`num_hands=2`	Setting name expected by MediaPipe.	The number supplied to that setting.
`model_asset_path=MODEL_PATH`	Setting name expected by MediaPipe.	Your variable containing the model address.

Rename the variable to see the difference

```
model_settings = mp_python.BaseOptions(
    model_asset_path=MODEL_PATH
)
options = mp_vision.HandLandmarkerOptions(
    base_options=model_settings,
    num_hands=2,
)
```

This smaller example illustrates the names only; keep VIDEO mode in your actual project. The setting on the left remains `base_options`. The value on the right can come from a variable with any sensible name.

Calls can span several lines between matching parentheses. The final comma in a multiline call is allowed. It separates inputs; it does not create another setting.

FOUNDATION 4

Lists, tuples, dictionaries, and indexing

The same square brackets have different meanings depending on the value being accessed.

```
tips = [8, 12, 16, 20]
tips[0]                     # 8: first item in a list

pair = ("Ga", "3_Ga.mp3")
pair[0]                     # "Ga": first item in a tuple

sounds = {3: ("Ga", "3_Ga.mp3")}
sounds[3]                   # Look up dictionary key 3
sounds[3][0]                # Look up 3, then take item 0
```

Container	What it means in this project
List: `[ ... ]`	An ordered collection: finger-tip numbers or detected hands.
Tuple: `( ... )`	A grouped pair such as a note name and its filename.
Dictionary: `{key: value}`	A lookup from a finger count to its note information.

List and tuple positions start at zero. Dictionary brackets select a key, which does not have to be a position. In `sounds[3]`, the 3 is the finger-count key.

```
name, filename = sounds[3]
# Same values as:
name = "Ga"
filename = "3_Ga.mp3"
```

Assigning to two names separates the two returned values. The camera uses the same pattern: `success, frame = cap.read()`.

`len(items)` counts items. `count in sounds_mapping` checks whether a key exists. `frame.shape[:2]` takes the first two items of the shape: height and width.

> **Three operations to distinguish**
> `hand[8]`: select one landmark. `hand[8].y`: read its y value. `len(result.hand_landmarks)`: count detected hands, not the 21 points inside each hand.

FOUNDATION 5

Indentation, loops, and functions

Spaces at the start of a line show which instructions belong together.

```
count = 0
for tip in [8, 12, 16, 20]:
    if hand[tip].y < hand[tip - 2].y:
        count += 1
```

The `for` statement takes one number at a time and saves it as tip. The indented comparison runs for each tip. The more deeply indented addition runs only when the comparison is True. A colon starts a block; use consistent indentation, normally four spaces.

```
while True:
    success, frame = cap.read()
    if not success:
        break

cap.release()
```

`while True` repeats until a break or error ends it. `not success` is True when success is False. `break` exits the nearest loop. `cap.release()` is outside this loop because its indentation returns to the left.

```
def draw_landmarks(frame, hand_landmarks):
    # Instructions belonging to the function go here.
    h, w = frame.shape[:2]

draw_landmarks(frame, hand)
```

`def` defines a function. Its input names are frame and hand_landmarks. Defining it prepares the instructions; calling it runs them. At this call, the detected hand is supplied as the second input, so it is called hand_landmarks inside the function.

> **Names can change at the function boundary**
> hand outside and `hand_landmarks` inside can refer to the same supplied hand. The names do not have to match. The first input goes into the first parameter, and the second goes into the second.

PROGRAM MAP

What runs once, and what repeats?

Defining the drawing helper does not start the camera or run the helper body.



When	Source lines	What happens
Once, at the start	1-32	Import tools, locate assets, and create lookup data.
Once, at the start	35-41	Define the drawing helper. Its body waits until it is called.
Once, before the loop	44-64	Prepare resources, sound, detector, camera, and initial values.
For every frame	66-111	Read, mirror, detect, count, draw, choose audio, and display.
For every detected hand	80-94	Count that hand and call the drawing helper.
After the loop or an error	113-119	Release the camera, model, windows, and audio.

The variables have different lifetimes

`landmarker` is reused for all frames. `previous_count` remembers the preceding frame. `count` is reset to zero for each new frame. `hand` temporarily refers to the hand currently being processed.

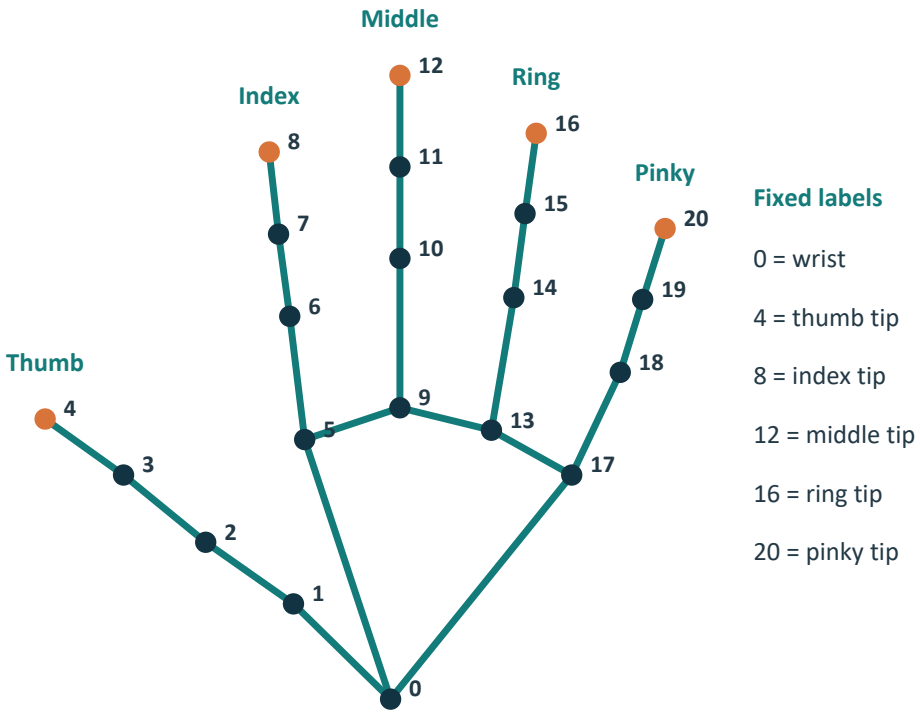
```
# Conceptual order, not extra source to paste:
prepare tools once
repeat for each camera photo:
    process this photo
    remember the count for the next photo
release tools when finished
```

Why the position of a line matters
Putting `count = 0` inside the hand loop loses the first hand's count. Putting it outside the frame loop makes totals accumulate across photos. The saved code resets it once per frame, before examining any hand.

MEDIAPIPE PICTURE

Learn the 21 hand-point numbers

A landmark is one estimated point on a hand. A landmark number identifies its position on the hand.



Schematic hand: the numbers identify points, not the number of raised fingers.

Part	Point numbers	Tip
Wrist	0	-
Thumb	1, 2, 3, 4	4
Index	5, 6, 7, 8	8
Middle	9, 10, 11, 12	12
Ring	13, 14, 15, 16	16
Pinky	17, 18, 19, 20	20

The detector estimates these positions for each visible hand. Your Python comparisons use the positions to count raised fingers. Landmark 8 always identifies the index tip; it is not a count of eight fingers.

MCP is a finger-base knuckle. PIP is the next major joint; DIP is the joint nearer the tip. For the four long fingers, the PIP number is the tip number minus two. The thumb uses a different joint layout.

IMAGE COORDINATES

What x and y actually contain

The detector gives positions relative to image width and height. Drawing uses pixel positions.



`x = 0` is the left edge and `x = 1` is the right edge. `y = 0` is the top and `y = 1` is the bottom. A point at x = 0.5, y = 0.25 is halfway across and one-quarter of the way down.

```
w = 640
h = 480
x = int(0.5 * w)     # 320 pixels from the left
y = int(0.25 * h)    # 120 pixels from the top
```

Multiply a relative x by the width and a relative y by the height. `int()` converts each result to a whole number for the drawing functions. The saved code does this for every hand point.

Point	Example y	Position
Index tip: 8	0.20	Higher in the picture.
Index PIP joint: 6	0.50	Lower in the picture.

The direction that often causes confusion
Smaller y means higher. That is why an upright raised index finger can satisfy `hand[8].y < hand[6].y`. Compare relative coordinates directly; convert to pixels when drawing.

SOURCE LINES 1-13

Import tools and locate the model

An import makes library names available. It does not start the camera or play a sound.

```
1  import os
2  import time
3
4  import cv2
5  import mediapipe as mp
6  import pygame
7  from mediapipe.tasks import python as mp_python
8  from mediapipe.tasks.python import vision as mp_vision
```

Line	What the statement gives us
1	`os`: functions for working with file paths.
2	`time`: a clock for elapsed-time measurements.
4	`cv2`: OpenCV camera, image conversion, drawing, and window functions.
5	`mediapipe as mp`: import MediaPipe and use mp as its shorter name.
6	`pygame`: the library whose mixer plays sound.
7	From mediapipe.tasks, import the part called python as mp_python.
8	From mediapipe.tasks.python, import vision as mp_vision.

```
12 folder = os.path.dirname(os.path.abspath(__file__))
13 MODEL_PATH = os.path.join(folder, "hand_landmarker.task")
```

For line 12, work from the innermost parentheses outward. `__file__` identifies this script; `abspath()` makes its address absolute; `dirname()` selects its folder. Line 13 joins that folder with the model filename.

```
# Example result, not your exact computer path:
folder = "C:/project"
MODEL_PATH = "C:/project/hand_landmarker.task"
```

`MODEL_PATH` holds text containing a file address. It is not the detector. The model is loaded later when the detector is created.

SOURCE LINES 15-32

Create the two lookup collections

These are ordinary Python data values; neither collection runs detection.

```
15  sounds_mapping = {
16      1: ("Sa", "1_Sa.mp3"),
17      2: ("Re", "2_Re.mp3"),
18      3: ("Ga", "3_Ga.mp3"),
19      4: ("Ma", "4_Ma.mp3"),
20      5: ("Pa", "5_Pa.mp3"),
21      6: ("Dha", "6_Dha.mp3"),
22      7: ("Ni", "7_Ni.mp3"),
23      8: ("Sa (upper)", "8_Sa_upper.mp3"),
24  }
```

Line 15 opens the sounds dictionary and line 24 closes it. Lines 16-23 each associate one finger-count key with a note-name and filename pair. All eight entries use the same structure.

```
sounds_mapping[5]       # ("Pa", "5_Pa.mp3")
sounds_mapping[5][0]    # "Pa"
5 in sounds_mapping    # True
9 in sounds_mapping    # False
```

```
26  HAND_CONNECTIONS = [
27      (0, 1), (1, 2), (2, 3), (3, 4),
28      (0, 5), (5, 6), (6, 7), (7, 8),
29      (5, 9), (9, 10), (10, 11), (11, 12),
30      (9, 13), (13, 14), (14, 15), (15, 16),
31      (13, 17), (0, 17), (17, 18), (18, 19), (19, 20),
32  ]
```

Line 26 opens the connections list and line 32 closes it. Each tuple is a pair of landmark indices. These are instructions for which points to connect when we draw.

Line	Connections described
27	Wrist through the thumb joints to its tip.
28	Wrist to index base, then through the index finger.
29	Index base to middle base, then through the middle finger.
30	Middle base to ring base, then through the ring finger.
31	Remaining palm edges and the pinky joints.

SOURCE LINES 35-41

Draw the skeleton: the complete helper

The helper receives the current image and the landmarks for one hand.

```
35  def draw_landmarks(frame, hand_landmarks):
36      h, w = frame.shape[:2]
37      points = [(int(lm.x * w), int(lm.y * h)) for lm in hand_landmarks]
38      for start, end in HAND_CONNECTIONS:
39          cv2.line(frame, points[start], points[end], (0, 255, 0), 2)
40      for x, y in points:
41          cv2.circle(frame, (x, y), 4, (0, 0, 255), -1)
```

Line	What happens
35	Define the function. Its two parameters receive the values supplied at a later call.
36	Get height and width from the first two items of frame.shape.
37	Create a list containing the pixel position of every landmark.
38-39	Take each connection pair and draw a green line, thickness 2.
40-41	Take each pixel position and draw a red filled circle, radius 4.

Expand line 37 into familiar steps

```
points = []
for lm in hand_landmarks:
    x = int(lm.x * w)
    y = int(lm.y * h)
    points.append((x, y))
```

This longer version does the same work as line 37's list comprehension. `append()` adds one (x, y) tuple to the list. After 21 landmarks, points contains 21 pixel positions in the same order.

In `for start, end in HAND_CONNECTIONS`, Python separates a pair such as (0, 1) into start = 0 and end = 1. `points[start]` then retrieves landmark 0's pixel position.

Colour and side effects
OpenCV colour tuples use BGR order: (0, 255, 0) is green; (0, 0, 255) is red. A circle thickness of -1 means filled. These drawing calls modify frame directly, so the helper needs no explicit return statement.

SOURCE LINES 44-57

Create one detector using named settings

Read this as: prepare model settings, prepare detector settings, build the detector.

```
44  cap = None
45  landmarker = None
46
47  try:
48      pygame.mixer.init()
49
50      # Create the detector once, before the webcam loop.
51      base_options = mp_python.BaseOptions(model_asset_path=MODEL_PATH)
52      options = mp_vision.HandLandmarkerOptions(
53          base_options=base_options,
54          running_mode=mp_vision.RunningMode.VIDEO,
55          num_hands=2,
56      )
57      landmarker = mp_vision.HandLandmarker.create_from_options(options)
```

Line	Plain-language meaning
44-45	Start both resource variables at None so cleanup can check whether they were assigned.
47	Start the try block. The finally block at the end will run when this block is left.
48	Initialize Pygame's audio mixer.
51	Create basic model settings using the filename in MODEL_PATH.
52-56	Create the hand-detector settings object and save it as options.
53	Supply the basic settings created on line 51. Left side: setting name. Right side: your variable.
54	Choose the predefined VIDEO mode value for a sequence of timestamped frames.
55	Allow at most two detected hands. This is a limit, not the actual detected count.
57	Call create_from_options(options); save the returned detector as landmarker.

Keep this outside the loop
The settings and model-loading work are done once. The existing detector is then asked to process many images. Recreating it for every frame adds unnecessary model setup work.

SOURCE LINES 58-70

Open the camera and start the loop

The camera read returns two values, so there are two names on the left.

```
58  cap = cv2.VideoCapture(0)
59  if not cap.isOpened():
60      raise RuntimeError("Cannot open the webcam. Close other camera apps and try again.")
61
62  previous_count = 0
63  start_time = time.monotonic()
64  timestamp_ms = -1
65
66  while True:
67      success, frame = cap.read()
68      if not success:
69          print("Could not read a frame from the webcam.")
70          break
```

Line	Plain-language meaning
58	Create a camera connection using device index 0, usually the default camera.
59-60	If opening failed, raise a RuntimeError with the supplied message. Cleanup still runs.
62	Remember zero as the previous frame's finger count.
63	Record a starting clock value.
64	Initialize the preceding timestamp to -1.
66	Begin repeating the frame-processing block.
67	Read one photo: success receives a Boolean; frame receives the image array.
68-70	If the read failed, print a message and break out of the loop.

Understand the clock

`time.monotonic()` is a clock for measuring differences. Its starting number is not a date or a count of seconds since you opened the program. Subtracting start_time from a later reading gives elapsed seconds.

```
# Invented clock readings:
start_time = 500.000
current_time = 500.034
elapsed = current_time - start_time   # About 0.034 s
```

Your milestone notes use `time.time()`. The saved program uses monotonic() so an adjustment to the computer's wall clock does not make elapsed time jump backward.

SOURCE LINES 72-80

Give each image to MediaPipe

Keep the BGR image for display; prepare a separate RGB image for the detector.

```
72  frame = cv2.flip(frame, 1)
73  rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
74  mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)
75  # Each frame must have a newer timestamp than the previous one.
76  timestamp_ms = max(timestamp_ms + 1, int((time.monotonic() - start_time) * 1000))
77  result = landmarker.detect_for_video(mp_image, timestamp_ms)
78
79  count = 0
80  for hand in result.hand_landmarks:
```

Line	What the call does
72	Mirror the camera image horizontally and save the returned image as frame.
73	Convert BGR channel order to RGB and save it separately as rgb_frame.
74	Create an mp.Image object. image_format describes the pixels; data supplies them.
76	Calculate elapsed milliseconds, ensuring a larger timestamp than before.
77	Call the detector with the image and its time. Save its returned result object.
79-80	Start the frame total at zero, then take each detected hand in turn.

Read the long timestamp expression in pieces

```
elapsed_seconds = time.monotonic() - start_time
elapsed_ms = int(elapsed_seconds * 1000)
timestamp_ms = max(timestamp_ms + 1, elapsed_ms)
```

`int()` removes the fractional part. `max(a, b)` returns the larger value. If elapsed_ms is 34 and the previous timestamp was 33, the new timestamp is 34. If both frames fall in the same millisecond, the +1 still makes the new timestamp larger.

> **What is inside result?**
> `result.hand_landmarks` is a list of hands. Each hand is a list of 21 landmark objects. Zero hands means an empty list. `len(result.hand_landmarks)` gives 0, 1, or 2 with this configuration.

SOURCE LINES 79-84

Count the four long fingers

A raised fingertip is above its PIP joint when the hand is upright.

```
79  count = 0
80  for hand in result.hand_landmarks:
81      # Four fingers: the tip is above the PIP joint when raised.
82      for tip in [8, 12, 16, 20]:
83          if hand[tip].y < hand[tip - 2].y:
84              count += 1
```

Line 79 resets the total once for this photo. Line 80 chooses a hand. Line 82 chooses each fingertip number in turn. Line 83 compares the tip's y with the PIP joint's y. Line 84 adds one only when that comparison is True.

Finger	tip	tip - 2	Actual comparison
Index	8	6	`hand[8].y < hand[6].y`
Middle	12	10	`hand[12].y < hand[10].y`
Ring	16	14	`hand[16].y < hand[14].y`
Pinky	20	18	`hand[20].y < hand[18].y`

```
# Example of a raised index finger:
tip_y = 0.20
joint_y = 0.50
tip_y < joint_y         # True: tip is higher

# Example of a folded index finger:
tip_y = 0.65
joint_y = 0.50
tip_y < joint_y         # False
```

Two hands contribute to the same total

Moment	count
Start of a new frame	0
After first hand with 4 raised fingers	4
After second hand with 2 raised fingers	6

Why we do not reset inside the hand loop
A reset for each hand would erase the first hand's contribution. This simple geometry also assumes visible, mostly upright fingers; unusual rotations and half-bent fingers can defeat the comparison.

SOURCE LINES 87-94

Count the thumb in either direction

This version determines sideways direction using the index and pinky bases.

```
87  if hand[5].x < hand[17].x:
88      if hand[4].x < hand[3].x:
89          count += 1
90  else:
91      if hand[4].x > hand[3].x:
92          count += 1
93
94  draw_landmarks(frame, hand)
```

Landmark	Meaning
5	Index-finger base
17	Pinky base
4	Thumb tip
3	Thumb joint just below the tip

Line 87 asks whether the index base is left of the pinky base in the image. Smaller x means farther left. For an upright palm facing the camera, this establishes which sideways direction is outward toward the thumb.

Palm arrangement	Thumb condition	What the code does
`hand[5].x < hand[17].x`	`hand[4].x < hand[3].x`	Lines 88-89 count a thumb extending left.
Opposite arrangement	`hand[4].x > hand[3].x`	Lines 90-92 count a thumb extending right.

The first condition chooses a direction. The inner condition checks extension in that direction. `else` belongs to the outer direction check, because their indentation levels match.

Line 94 calls `draw_landmarks(frame, hand)`. The helper draws that particular hand onto the existing camera image. The call occurs once per detected hand, after counting it.

Difference from your student guide
The guide suggests choosing the thumb direction from a Left/Right label. This saved file uses the relative positions of landmarks 5 and 17 instead. It avoids reading handedness, but still assumes the upright palm pose; it is not a general pose classifier.

SOURCE LINES 97-104

Play a sound only when the count changes

The previous_count variable prevents a held gesture from restarting the clip every frame.

```
97  if count != previous_count:
98      if count in sounds_mapping:
99          name, filename = sounds_mapping[count]
100         pygame.mixer.music.load(os.path.join(folder, "sounds_swaras", filename))
101         pygame.mixer.music.play()
102     else:
103         pygame.mixer.music.stop()
104     previous_count = count
```

Line	Meaning
97	Continue into this block only if count differs from previous_count.
98	Check whether count is a key in the note dictionary.
99	Separate the selected tuple into note name and audio filename.
100	Join the project folder, sounds folder, and filename; prepare that music file.
101	Start playback. The camera loop can continue while audio plays.
102-103	For unmapped counts 0, 9, and 10, stop the music stream.
104	Remember this count, including silent counts, for the next frame.

```
# If count is 3:
name, filename = sounds_mapping[3]
# name is "Ga"; filename is "3_Ga.mp3"
```

Pygame's music controller uses one music stream. A new accepted count can replace the previous note. A held gesture is not repeated automatically: the short clip finishes while the count stays the same.

What makes the same note play again?
Change to a different count and return. For example: 3 plays Ga, 0 stops it and stores zero, then 3 differs from zero and plays Ga again. Fast transitions may play intermediate counts; this simple version has no gesture-stability timer.

SOURCE LINES 106-111

Show the count and listen for q

The label describes the current gesture, even after its short audio clip has finished.

```
106  name = sounds_mapping[count][0] if count in sounds_mapping else "Silence"
107  cv2.putText(frame, f"Fingers: {count}  Swara: {name}", (20, 40),
108              cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
109  cv2.imshow("Swara Player - press q to quit", frame)
110  if cv2.waitKey(1) & 0xFF == ord("q"):
111      break
```

Line 106 is a conditional expression: use the selected note's name if the dictionary contains count; otherwise use the text Silence. The [0] takes the first item of the selected (name, filename) tuple.

putText input	Meaning
`frame`	Image that receives the writing.
`f"Fingers: {count} Swara: {name}"`	An f-string: insert the variable values inside braces.
`(20, 40)`	Text baseline begins 20 pixels right and 40 pixels down from the top-left.
`cv2.FONT_HERSHEY_SIMPLEX`	Select a predefined font.
`0.8`	Font scale.
`(0, 255, 0)`	Green in BGR order.
`2`	Stroke thickness in pixels.

Lines 107-108 are one call spread over two source lines. Line 109 shows the modified frame using the window title in quotes. Reusing the title updates the same window.

```
key_number = cv2.waitKey(1) & 0xFF
if key_number == ord("q"):
    break
```

This expanded form explains lines 110-111. waitKey(1) processes window events and briefly checks the keyboard. The bit mask 0xFF keeps the lowest eight bits, giving a value from 0 to 255. ord("q") is 113. Equality to 113 means lowercase q was pressed, so break leaves the frame loop.

SOURCE LINES 113-119

Close resources after success or failure

A finally block performs cleanup; it does not hide an error.

```
113 finally:
114     if cap is not None:
115         cap.release()
116     if landmarker is not None:
117         landmarker.close()
118     cv2.destroyAllWindows()
119     pygame.mixer.quit()
```

Line	Meaning
113	Run this block when execution leaves the corresponding try block.
114	Check that cap was assigned an object rather than left as None.
115	Release the webcam connection.
116	Check that a detector was successfully created.
117	Close the detector and its resources.
118	Close OpenCV's windows.
119	Shut down the Pygame audio mixer.

The initial None values on lines 44-45 let cleanup handle incomplete setup. If model creation fails before cap is assigned, cap remains None and the camera release is skipped.

```
try:
    # Set up the tools and run the camera loop.
    ...
finally:
    # Release whichever tools were created.
    ...
```

This is the program's overall structure, not additional code. Pressing q breaks the loop; reaching the end of try then runs finally. A Python exception also runs finally before the error continues to be reported.

None and not are different ideas
`cap is not None` asks whether cap contains something other than the special None value. `not success` asks whether a Boolean is False. Neither expression opens or closes anything by itself.

In the saved version, use lowercase q with the camera window focused. The code does not implement a separate close-window-button exit check or an audio mute key.

MILESTONE PRACTICE

First make only the webcam work

Use a separate practice file if you want to build the program from the beginning.

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    success, frame = cap.read()
    if not success:
        break

    frame = cv2.flip(frame, 1)
    cv2.imshow("My Webcam", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Use this as a small learning example, not as a replacement for all the final program's setup and cleanup. Run it with the same Python environment that successfully imports cv2.

Test	Expected result
Run the file	A live camera window appears if the camera is available.
Move sideways	The displayed image behaves like a mirror.
Focus the window; press lowercase q	The loop ends and the window closes.

Do not add hand detection until this works. At this stage, the image contains no hand dots, no count, and no sound. If the window does not open, check camera access, other camera applications, and the interpreter shown on page 26.

A useful experiment
Change the window title text. Then change it back. This lets you see which input controls the window name without changing any camera logic.

MILESTONES 2-4

Add detection, then drawing

Each addition has a location. The full numbered source on pages 28-30 shows their final positions.

Milestone 2: setup before the loop

```
base_options = mp_python.BaseOptions(model_asset_path=MODEL_PATH)
options = mp_vision.HandLandmarkerOptions(
    base_options=base_options,
    running_mode=mp_vision.RunningMode.VIDEO,
    num_hands=2,
)
landmarker = mp_vision.HandLandmarker.create_from_options(options)
```

Also add the imports on source lines 5, 7, and 8 and the model path on lines 12-13. The included model must exist. At this checkpoint the detector is created, but the webcam is still an ordinary image.

Milestone 3: detection inside the loop

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)
result = landmarker.detect_for_video(mp_image, timestamp_ms)
print(len(result.hand_landmarks))
```

Place this after mirroring the frame and after computing timestamp_ms as explained on page 16. The printed hand count should be 0, 1, or 2. The print is a temporary checkpoint and is not in the saved final file.

Milestone 4: draw after detection, before display

```
for hand in result.hand_landmarks:
    draw_landmarks(frame, hand)
```

Define the connections and helper before the loop. This call draws on the original BGR frame; show that frame with imshow afterward. In the final file, the same hand loop also counts fingers before calling the helper.

> **After these checkpoints**
> Add counting, note lookup, and sound as described on pages 17-20. Keep detector creation outside the frame loop, and call landmarker.close() in the final cleanup block.

WORKED EXAMPLE

Follow the values through several frames

The count changes with each processed photo; previous_count carries information into the next photo.

Frame	previous_count before	count now	Audio action
1	0	3	Load Ga and play it.
2	3	3	No new audio call. Existing clip continues or finishes.
3	3	3	No restart while the gesture is held.
4	3	0	Stop the music stream.
5	0	3	Load Ga and play it again.
6	3	9	Stop; count 9 has no dictionary entry.
7	9	5	Load Pa and play it.
8	5	10	Stop; count 10 has no dictionary entry.

After each changed count, line 104 saves that new value as previous_count. When the count is unchanged, the old and new values already agree, so the sound block is skipped.

Trace one frame with two hands

```
count = 0
# First hand contributes 4:
count = 4
# Second hand contributes 2:
count = 6
name, filename = sounds_mapping[6]
# name = "Dha", filename = "6_Dha.mp3"
```

Those assignments illustrate the arithmetic; the actual program obtains each contribution through landmark comparisons. The total is six because count was reset once before both hands, not between them.

What you see and what you hear are different
The display can keep saying Ga after the clip has ended because it describes your current finger count. The program does not loop that clip or change the name to Silence merely because playback finished.

PRACTICE WITH ANSWERS

Check whether you can read the syntax

Try each prediction before looking at the answer table.

1. What is stored in name after `name = sounds_mapping[4][0]`?

2. A landmark has x = 0.25 and y = 0.5 in an image 640 pixels wide and 480 pixels high. What pixel position will be drawn?

3. The index tip has y = 0.30 and its PIP joint has y = 0.55. Does the four-finger test add one?

4. previous_count is 2 and count is also 2. Does the code call music.play() again?

5. No hands are detected. How many times does `for hand in result.hand_landmarks` run, and what is count?

6. In `num_hands=2`, which part is MediaPipe's setting name, and which part supplies the value?

7. What happens if `count = 0` is moved inside the loop over hands?

Question	Answer and reason
1	Ma. Key 4 selects the pair; [0] selects its first item.
2	(160, 240). Multiply x by width and y by height.
3	Yes. 0.30 < 0.55 is True, so the tip is higher in the image.
4	No. The outer count != previous_count comparison is False.
5	Zero times. count stays at the zero assigned before the hand loop.
6	num_hands is the fixed setting name; 2 is the supplied value.
7	The next hand resets the total, losing the preceding hand's contribution.

A practical reading habit
Point to the variable on the left of = and say its value aloud. For a call, name its owner, action, inputs, and returned value. This is more useful than memorizing the entire line as one command.

RUN AND TROUBLESHOOT

Use the Python that has your modules

The program can be correct while a different Python installation is missing its libraries.

On this computer, the prepared `Start-Swara-Player.cmd` in the parent outputs folder launches the correct environment. For a separate setup, use Python 3.10, 3.11, or 3.12 and keep installation and execution in that same environment.

```
python -m pip install -r requirements.txt
python main.py

# To identify the Python used by this terminal:
python -c "import sys; print(sys.executable)"
```

These commands assume python points to a supported interpreter. The current requirements use MediaPipe 0.10.14, OpenCV contrib 4.11.0.86, Pygame 2.6.1, and NumPy 1.26.4. The prepared environment was tested with Python 3.12.

```
project/
    main.py
    hand_landmarker.task
    requirements.txt
    sounds_swaras/
        1_Sa.mp3 ... 8_Sa_upper.mp3
```

Symptom	First thing to check
No module named cv2	The active Python environment. cv2 is supplied by the OpenCV package.
Model cannot open	The .task file is beside main.py in this version. It is included, not automatically downloaded.
Camera unavailable	Camera permissions, other camera apps, or the device index on line 58.
Dots near the top-left	Landmark fractions must be multiplied by width and height.
Sound constantly restarts	Keep the count != previous_count condition and update previous_count.
Unexpected note or count	Keep palms upright, improve light, and inspect the tip-versus-joint comparisons.

QUICK REFERENCE

A small dictionary of code words

Read this page when a word stops you from following the next statement.

Word or symbol	Meaning in this program
Variable	A name referring to a stored value, such as count.
Library / module	Reusable code imported with a name such as cv2.
Alias	A shorter imported name: mediapipe as mp.
Function	A callable operation with a name and inputs.
Method	A callable operation attached to an object, such as cap.read().
Object	A value that can hold information and offer methods.
Class	The type used to create objects, such as BaseOptions.
Attribute	Named information read with a dot, such as landmark.y.
Argument	A value supplied when you call a function.
Parameter	An input name in a function's definition.
Return value	The result a call gives back for later use.
Index	An item's position in a sequence; positions start at zero.
Dictionary key	A value used for lookup, such as finger count 3.
Frame	One camera image, stored as an array of pixel values.
Landmark	One detected hand point with coordinate values.
Timestamp	A frame's time value, in milliseconds for this detector call.
Boolean	True or False, often returned by a comparison.
None	The special value used here before a resource object exists.
`()` / `[]` / `.`	Call or group / access sequence items or dictionary keys / access a named member.

To check a fixed library name, consult its documentation. To check one of your own variable names, look for the assignment that last gave it a value.

SOURCE APPENDIX

Complete source: setup and drawing

Original lines 1-41. Line numbers in the left margin are reference labels, not Python code.

```
1  import os
2  import time
3
4  import cv2
5  import mediapipe as mp
6  import pygame
7  from mediapipe.tasks import python as mp_python
8  from mediapipe.tasks.python import vision as mp_vision
9
10
11 # Keep the model and sounds beside this script.
12 folder = os.path.dirname(os.path.abspath(__file__))
13 MODEL_PATH = os.path.join(folder, "hand_landmarker.task")
14
15 sounds_mapping = {
16     1: ("Sa", "1_Sa.mp3"),
17     2: ("Re", "2_Re.mp3"),
18     3: ("Ga", "3_Ga.mp3"),
19     4: ("Ma", "4_Ma.mp3"),
20     5: ("Pa", "5_Pa.mp3"),
21     6: ("Dha", "6_Dha.mp3"),
22     7: ("Ni", "7_Ni.mp3"),
23     8: ("Sa (upper)", "8_Sa_upper.mp3"),
24 }
25
26 HAND_CONNECTIONS = [
27     (0, 1), (1, 2), (2, 3), (3, 4),
28     (0, 5), (5, 6), (6, 7), (7, 8),
29     (5, 9), (9, 10), (10, 11), (11, 12),
30     (9, 13), (13, 14), (14, 15), (15, 16),
31     (13, 17), (0, 17), (17, 18), (18, 19), (19, 20),
32 ]
33
34
35 def draw_landmarks(frame, hand_landmarks):
36     h, w = frame.shape[:2]
37     points = [(int(lm.x * w), int(lm.y * h)) for lm in hand_landmarks]
38     for start, end in HAND_CONNECTIONS:
39         cv2.line(frame, points[start], points[end], (0, 255, 0), 2)
40     for x, y in points:
41         cv2.circle(frame, (x, y), 4, (0, 0, 255), -1)
```

The statements match the current simplified main.py. Blank lines and comments are included so the line references elsewhere in this guide stay exact.

SOURCE APPENDIX

Complete source: camera and detection

Original lines 42-80. Continue directly after the previous page.

```
42
43
44  cap = None
45  landmarker = None
46
47  try:
48      pygame.mixer.init()
49
50      # Create the detector once, before the webcam loop.
51      base_options = mp_python.BaseOptions(model_asset_path=MODEL_PATH)
52      options = mp_vision.HandLandmarkerOptions(
53          base_options=base_options,
54          running_mode=mp_vision.RunningMode.VIDEO,
55          num_hands=2,
56      )
57      landmarker = mp_vision.HandLandmarker.create_from_options(options)
58      cap = cv2.VideoCapture(0)
59      if not cap.isOpened():
60          raise RuntimeError("Cannot open the webcam. Close other camera apps and try again.")
61
62      previous_count = 0
63      start_time = time.monotonic()
64      timestamp_ms = -1
65
66      while True:
67          success, frame = cap.read()
68          if not success:
69              print("Could not read a frame from the webcam.")
70              break
71
72          frame = cv2.flip(frame, 1)
73          rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
74          mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)
75          # Each frame must have a newer timestamp than the previous one.
76          timestamp_ms = max(timestamp_ms + 1, int((time.monotonic() - start_time) * 1000))
77          result = landmarker.detect_for_video(mp_image, timestamp_ms)
78
79          count = 0
80          for hand in result.hand_landmarks:
```

The statements match the current simplified main.py. Blank lines and comments are included so the line references elsewhere in this guide stay exact.

SOURCE APPENDIX

Complete source: counting and sound

Original lines 81-119. Continue directly after the previous page.

```
            # Four fingers: the tip is above the PIP joint when raised.
            for tip in [8, 12, 16, 20]:
                if hand[tip].y < hand[tip - 2].y:
                    count += 1

            # Thumb: use the index and pinky bases to find the outward direction.
            if hand[5].x < hand[17].x:
                if hand[4].x < hand[3].x:
                    count += 1
            else:
                if hand[4].x > hand[3].x:
                    count += 1

            draw_landmarks(frame, hand)

        # Play only when the count changes, so a held gesture does not stutter.
        if count != previous_count:
            if count in sounds_mapping:
                name, filename = sounds_mapping[count]
                pygame.mixer.music.load(os.path.join(folder, "sounds_swaras", filename))
                pygame.mixer.music.play()
            else:
                pygame.mixer.music.stop()
            previous_count = count

        name = sounds_mapping[count][0] if count in sounds_mapping else "Silence"
        cv2.putText(frame, f"Fingers: {count}  Swara: {name}", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
        cv2.imshow("Swara Player - press q to quit", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

finally:
    if cap is not None:
        cap.release()
    if landmarker is not None:
        landmarker.close()
    cv2.destroyAllWindows()
    pygame.mixer.quit()
```

The statements match the current simplified main.py. Blank lines and comments are included so the line references elsewhere in this guide stay exact.

SOURCES AND SCOPE

Where these explanations come from

This guide explains your saved implementation, including its small differences from the assignment hints.

Primary code: the local ERC-26_CV_Week4/main.py, 119 lines, as read on 29 October 2026. The complete numbered copy appears on pages 28-30. No statements in that file were changed to produce this PDF.

Learning context: the supplied Week 1 Python Foundations notes; Week 2 NumPy and Matplotlib notes; Week 3 OpenCV notes; CV_Week4_Notes.pdf; and the student guide pasted into the chat. These provide the course progression, while the saved code defines the exact statements explained here.

Official references - click a title

Python: control flow and functions

if, for, break, def, and function arguments.

Python: lists, tuples, and dictionaries

Collections, indexing, and dictionary operations.

Python: file path operations

abspath, dirname, and join.

Python: monotonic clock

Elapsed-time clock behavior.

OpenCV: camera capture and display

VideoCapture, read, imshow, waitKey, and release.

OpenCV: drawing functions

Lines, circles, colours, and text.

MediaPipe: Hand Landmarker Python guide

Model settings, image preparation, detection, and outputs.

MediaPipe: hand model and landmarks

Landmark numbering and hand-task overview.

Pygame: music playback

Preparing, playing, and stopping the music stream.

Documentation may show examples from a newer release. The code in this PDF follows the installed project versions listed on page 26. Hardware behavior still depends on your webcam, lighting, and audio output.

Source fingerprint (SHA-256): 7adbc817b092589d26388873e767200f32a8b06b87a5421d7e23a748412528b7